

Building the Terminal Feature

How the fake in-browser shell on the portfolio site works, end to end.

The terminal isn't simulating a real shell — it's a styled text input plus a tiny bit of JavaScript that reacts to one exact string. Everything below walks through the actual pieces, in the order they run.

1 The HTML skeleton

index.html

```
<div class="terminal" id="terminal">
  <div class="terminal-output" id="terminal-output"></div>
  <div class="terminal-line">
    <span class="prompt">guest@rafi:~$</span>
    <input type="text" id="terminal-input" autocomplete="off" spellcheck="false" autofocus>
  </div>
</div>
```

Three parts:

- `terminal-output` — an empty box that command history gets appended into
- `prompt` — static decorative text (`guest@rafi:~$`), never changes
- `terminal-input` — a real `<input>`. This is the actual thing the user types into

There's no fake "typing" animation or terminal emulator library — it's a plain text field styled with CSS to look like a shell prompt.

2 Grabbing the elements

terminal.js

```
const terminalOutput = document.getElementById('terminal-output');
const terminalInput = document.getElementById('terminal-input');
```

Straight lookups by the IDs from the HTML above. Nothing fancy — just handles to the DOM nodes we'll manipulate.

3 A helper to print a line

```
function printLine(text) {
  const line = document.createElement('div');
  line.textContent = text;
  terminalOutput.appendChild(line);
  terminalOutput.scrollTop = terminalOutput.scrollHeight;
}
```

Every time we want to show a line of "output," we create a new `<div>`, set its text, and append it into `terminal-output`.

`terminalOutput.scrollTop = terminalOutput.scrollHeight` is the auto-scroll trick: it jumps the scroll position to the very bottom so the newest line is always visible. This matters because `.terminal-output` has `max-height: 150px; overflow-y: auto;` in the CSS — once history exceeds that height, it becomes a scrollable box, so without this line new output would silently append below the visible area.

4 Listening for Enter

```
terminalInput.addEventListener('keydown', (e) => {
  if (e.key !== 'Enter') return;

  const command = terminalInput.value.trim();
  terminalInput.value = '';
  if (!command) return;

  printLine(`guest@rafi:~$ ${command}`);

  if (command === 'cd more') {
    printLine('cd: more.html');
    location.href = 'more.html';
  } else {
    printLine(`command not found: ${command}`);
  }
});
```

Broken down:

- `keydown` fires on every key pressed, so the first line filters for `Enter` only — anything else does nothing, which is how normal typing doesn't trigger the command logic on every keystroke.
- Once Enter is hit: read `.value`, `trim()` whitespace off the ends, then immediately clear the input back to empty — just like a real shell resets the line after you press Enter.

- `if (!command) return;` — if the trimmed string is empty (user just hit Enter on nothing), bail out early so it doesn't spam a blank line into history.
- `println(`guest@rafi:~$ ${command}`)` echoes what was typed as a history line, with the prompt prefixed — this is what makes it look like a transcript rather than a live input.
- Then a plain string comparison: if the command is exactly `"cd more"`, print a fake `cd:` confirmation line and navigate with `location.href = 'more.html'`. That's the entire "routing" logic — just a normal browser navigation to a static HTML file, dressed up to look like a shell command.
- Anything else falls into the `else` branch and prints `command not found: <whatever they typed>`, mimicking how a real shell responds to an unrecognized command.

5 Auto-focus on click

```
document.addEventListener('click', () => terminalInput.focus());
```

Clicking anywhere on the page refocuses the input, so the user can start typing without having to click precisely inside the small input box first.

The second copy: `more.html`

`more.html` uses an almost identical script, `terminal-more.js` — same structure, same five steps — except it checks for the command `"cd .."` and navigates back with `location.href = 'index.html'` instead. It's copy-pasted rather than shared between the two pages, so any fix or feature added to one won't automatically apply to the other.